KICKS.AI — Product Management Document (PMD) & Technical Manual

Comprehensive System Architecture, API Specification & Deployment Guide | Version 2.0

Project Name: KICKS AI Sneaker Boutique	**Version:** 2.0.0 (Production)
Telegram Bot: @Eg_chatbot (t.me/Eg_chatbot)	**Framework:** Flask + Telegram Async Bot
Backend API: Python / Gunicorn / SQLite	**Target Cloud:** Render.com / Railway.app

1. Executive Overview & Strategic Vision

KICKS.AI is an integrated, next-generation e-commerce ecosystem tailored for luxury and retro sneakers. The system merges a high-performance web storefront with an automated, 24/7 Telegram Assistant (@Eg_chatbot) to deliver a seamless conversational commerce experience across mobile and web channels.

Primary Product Goals:

- **Conversational Commerce:** Enable complete discovery, size selection, and order placement inside Telegram.
- **Database Grounded AI:** Ground all product answers strictly on SQLite database inventory to eliminate hallucinations.
- **Unified Order Synchronization:** Synchronize orders across web and Telegram using customer phone numbers.
- **Instant 1-Click Booking:** Minimize customer friction with pre-filled details and real-time inventory updates.

2. System Core Feature Matrix

Feature Module	Description & Capability	Technical File
Web Storefront	Glassmorphism responsive UI, brand filter chips, instant order modal, toast feedback.	templates/index.html static/js/app.js
Telegram Bot	Live polling via @Eg_chatbot, rich photo cards, inline size selector, size guide, order tracking.	bot.py
Web Telegram Simulator	Web preview of Telegram app (/telegram) with docked 2x2 persistent custom keyboard.	templates/telegram.html static/js/telegram_sim.js
AI Concierge Engine	Budget query parsing ('under $200'), sizing advice, standard out-of-stock messages.	app.py (/api/chat)
Order Management	Stock auto-decrementing, phone-matched lookup, status updates (Pending -> Shipped).	app.py (/api/orders) models.py

3. System Architecture & Database Schemas

The architecture follows a decoupled client-server pattern. The Flask REST API serves as the centralized backend, managing database state and responding to both Web UI requests and Telegram Bot Worker polling.

Database Schema Models (models.py):

Model Name	Key Attributes & Datatypes	Relationships & Constraints
Product	id (Int, PK) name (String) brand (String) price (Float) stock (Int) sizes (Text/JSON) image_url (Text) description (Text)	Referenced in Order.product_id Check stock >= quantity
Order	id (Int, PK) user_id (Int, Nullable) customer_name (String) customer_phone (String) shipping_address (Text) product_id (Int, FK) size (String) quantity (Int) total_price (Float) status (String) source (String)	FK -> Product.id FK -> User.id Indexed by customer_phone for cross-platform sync
User	id (Int, PK) name (String) email (String, Unique) password_hash (String) role (String: 'customer'/'admin') reset_token (String)	Has many Orders Role-based access control

4. How It Works: Technical Workflows

A. Telegram Bot Photo Card Catalog Workflow:

1. User clicks 'Browse Catalog' or sends /catalog to @Eg_chatbot.
2. bot.py issues GET request to /api/products.
3. bot.py loops through products and invokes send_photo with photo=p['image_url'], caption with price/stock/sizes, and an inline [■ Order] button.
4. Clicking [■ Order] opens inline size selection keyboard [US 8] [US 9] [US 10].
5. Selecting a size prompts user for Name, Phone, Address.

6. bot.py submits POST to /api/orders, decrements database stock, and sends instant order confirmation #ID card.

B. Cross-Platform Phone-Matched Order Sync Engine:

Orders placed on Web or Telegram both capture customer_phone. When queried via GET /api/orders?phone=, the engine sanitizes digits via regex re.sub(r'\D', '', phone) and returns matching orders, displaying status across all channels.

5. Cloud Infrastructure & Render.com Blueprint

The application is pre-configured with render.yaml for 1-click automated cloud deployment on Render.com.

render.yaml Specification:

```
services:
  - type: web
    name: sneaker-store-backend
    env: python
    buildCommand: pip install -r requirements.txt
    startCommand: gunicorn app:app
    envVars:
      - key: SECRET_KEY
        generateValue: true
      - key: TELEGRAM_BOT_TOKEN
        sync: false
      - key: TELEGRAM_BOT_USERNAME
        value: Eg_chatbot

  - type: worker
    name: sneaker-telegram-bot
    env: python
    buildCommand: pip install -r requirements.txt
    startCommand: python bot.py
    envVars:
      - key: TELEGRAM_BOT_TOKEN
        sync: false
      - key: BACKEND_API_URL
        fromService:
          type: web
          name: sneaker-store-backend
          property: host
          path: /api
```

Environment Variables Configuration Table:

Variable Name	Required Value / Example	Service Scope
TELEGRAM_BOT_TOKEN	8646676784:AAG6ICve37OJ0Mz0JOAwTo0Q-KvGtYZykGs	Web & Worker
TELEGRAM_BOT_USERNAME	Eg_chatbot	Web Service
BACKEND_API_URL	https://.onrender.com/api	Worker Service
OPENAI_API_KEY	sk-svcacct-...	Web & Worker
GEMINI_API_KEY	AQ.Ab8RN6...	Web & Worker

6. Maintenance, Error Resilience & Security

- **Photo Message Callback Resilience:** bot.py uses safe_edit_or_reply() to safely delete photo cards and reply with fresh text, avoiding Telegram BadRequest: There is no text in the message to edit.
- **Markdown Sanitization:** All dynamic product descriptions pass through escape_markdown() to prevent entity parsing crashes.
- **Multi-Worker Concurrency:** Production WSGI deployment uses gunicorn app:app to handle multi-user traffic.
- **Data Persistence:** SQLite database sneakerstore.db maintains total order history, stock allocations, and user accounts.